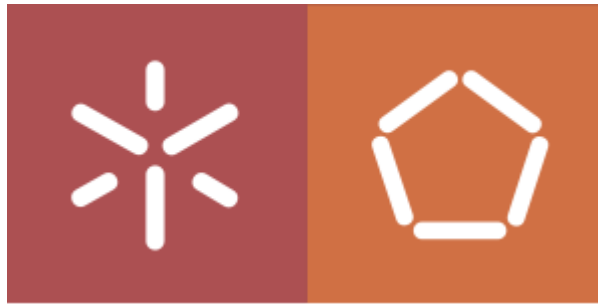




Universidade do Minho
Escola de Engenharia

Licenciatura em Engenharia Informática

Computação Gráfica

Grupo 23

A104619, Gonçalo da Silva Carmo

A104438, Gonçalo Filipe Duarte Barbosa

A106799, Jorge Gabriel Sá Barbosa

Segunda fase da componente prática, março de 2026

Introdução	3
Objetivo	3
Sistema de Gestão de Estado e Variáveis	3
Estruturas de Configuração Global	3
Armazenamento e Pre-Caching da Cena	4
O Parser: Motor de Interpretação XML	4
Carregamento de Ambiente (loadScene)	4
Parsing Recursivo de Grupos (parseGroup)	4
Robustez de Caminhos	4
Arquitetura do Grafo de Cena e Transformações	5
Caso Prático: Saturno e os seus Anéis	5

Introdução

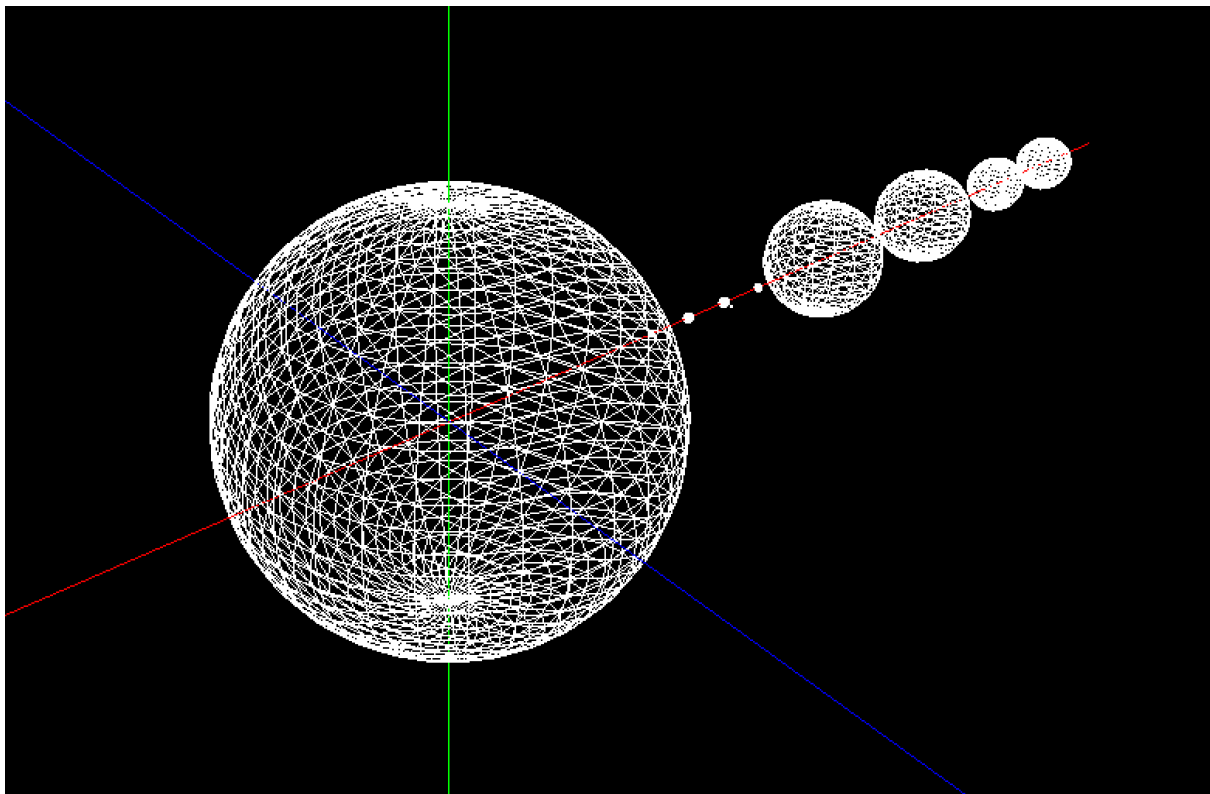
Nesta segunda fase do projeto de Computação Gráfica, o desenvolvimento focou-se exclusivamente na evolução do motor de visualização (engine). O motor deixou de ser um visualizador estático para se tornar um sistema dinâmico baseado em Grafos de Cena, capaz de interpretar e renderizar hierarquias complexas de objetos definidas através de ficheiros XML.

Objetivo

Representação do Sistema Solar

O desafio prático consistiu na modelação de um Sistema Solar. Para garantir a clareza visual, foram tomadas decisões de design fundamentais:

- **Escalabilidade Adaptada:** Utilizaram-se escalas não reais para que planetas e astros fossem perfeitamente perceptíveis, contornando a escala massiva do espaço real.
- **Hierarquia Orbital:** A estrutura de grupos permitiu organizar o Sol como centro e os planetas como subgrupos, facilitando a gestão de posições relativas



Sistema de Gestão de Estado e Variáveis

Um dos pontos críticos desta fase foi a criação de um sistema robusto para armazenar as configurações da cena. O motor não se limita a desenhar; ele mantém um estado persistente em memória.

Estruturas de Configuração Global

Para evitar o uso de variáveis soltas, foram criadas estruturas de dados que centralizam o estado da aplicação:

- Window Settings: Armazena as dimensões da janela, permitindo que o motor ajuste o *viewport dinamicamente*.
- Camera Settings: Guarda a posição (pos), foco (lookAt), orientação (up) e projeção (fov, near, far).
- Conversão de Coordenadas: No carregamento, o sistema converte as coordenadas Cartesianas do XML para o sistema Polar (camPos), permitindo que a interatividade (rotação orbital) seja fluida e contínua a partir da posição inicial.

Armazenamento e Pre-Caching da Cena

A cena é carregada integralmente para a memória antes do primeiro frame ser desenhado.

- Contentores Dinâmicos: Utilizaram-se `std::vector` para permitir que a cena tenha um número ilimitado de grupos, transformações e modelos.
- Pre-Caching de Modelos: Através da função `vectorize`, os ficheiros .3d são lidos e os seus milhares de vértices são armazenados em vetores de pontos dentro de cada grupo. Isto elimina a necessidade de acesso ao disco durante a renderização, garantindo uma performance de *frames* elevada.

O Parser: Motor de Interpretação XML

O Parser é a ponte entre o ficheiro de configuração e as estruturas de dados em memória. Utilizando a biblioteca TinyXML-2, o processo de leitura foi dividido em níveis de especificidade.

Carregamento de Ambiente (loadScene)

A função principal percorre o nó raiz <world>. Ela é responsável por identificar as secções globais (window e camera) e extrair os atributos necessários através de métodos como *QueryIntAttribute* e *FloatAttribute*. Esta fase garante que o motor de renderização conhece os limites do volume de visualização antes de começar a processar objetos.

Parsing Recursivo de Grupos (parseGroup)

Esta é a função mais sofisticada do motor. Ela processa cada bloco <group> seguindo uma lógica rigorosa:

1. **Transformações:** O parser identifica as tags *translate*, *rotate* e *scale*. Para cada uma, extrai os atributos x, y, z e angle e armazena-os numa estrutura *Transform*. A ordem de leitura é preservada, pois é fundamental para o resultado final da matriz de transformação.
2. **Modelos:** Ao encontrar a tag <models>, o parser itera sobre cada <model>, extrai o atributo file e delega o carregamento geométrico.
3. **Recursividade:** Sempre que o parser encontra um novo <group> dentro do atual, ele chama-se a si próprio. Isto cria uma estrutura em árvore onde cada nó filho herda o contexto do pai, permitindo aninhamentos complexos (ex: Sol -> Terra -> Lua).

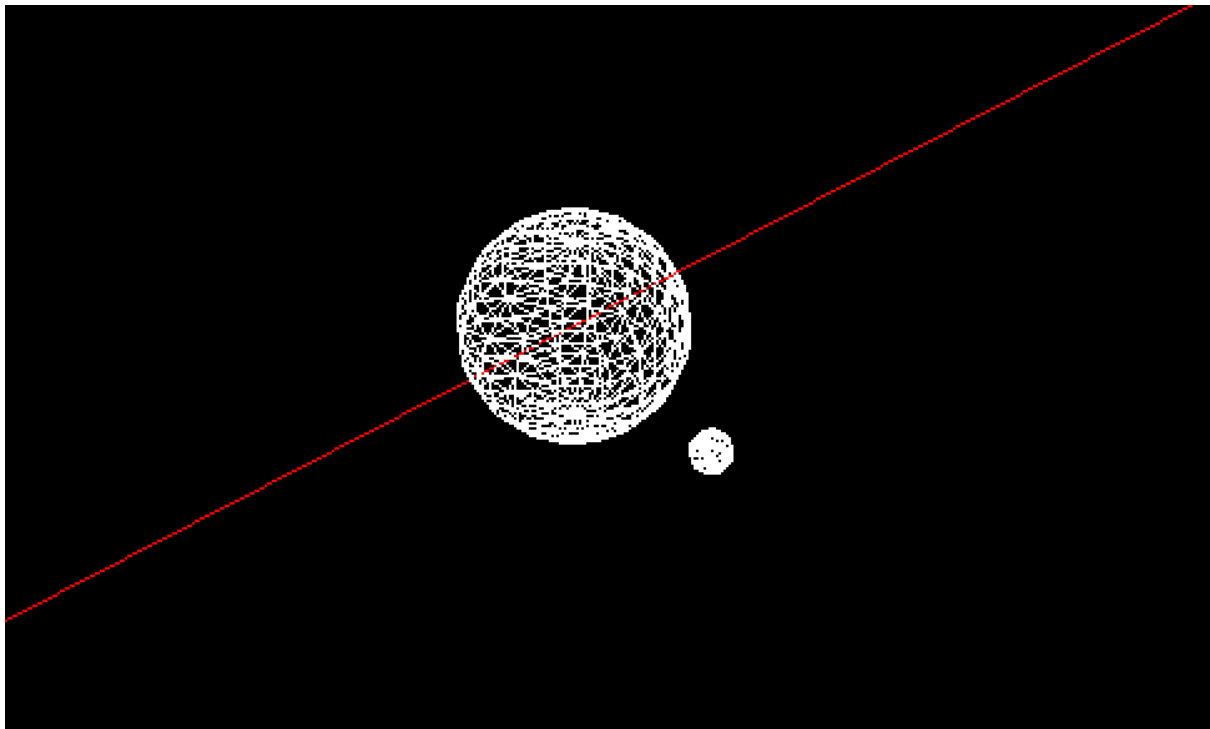
Robustez de Caminhos

O parser inclui lógica de verificação de caminhos. Se um ficheiro de modelo não for encontrado no caminho relativo direto, o motor tenta automaticamente a diretoria superior (../), garantindo que o projeto funcione independentemente da estrutura de pastas de compilação (ex: pastas build).

Arquitetura do Grafo de Cena e Transformações

A hierarquia de grupos permite a herança de transformações. Através das funções *glPushMatrix()* e *glPopMatrix()* no motor de desenho (*drawGroup*):

- As transformações aplicadas a um pai (ex: órbita da Terra) acumulam-se nos filhos (ex: Lua).
- A ordem das transformações no XML define se um astro executa uma rotação própria (Translação -> Rotação) ou um movimento orbital (Rotação -> Translação).



Conclusão

A Fase 2 elevou o projeto de um simples gerador de formas para um motor de renderização estruturado. A combinação de um sistema de armazenamento eficiente com um parser recursivo e robusto permitiu a criação de cenas complexas e hierárquicas, estabelecendo a base técnica necessária para as fases de animação e iluminação que se seguirão.